Telecom Customer Churn Analysis Using Python

WHAT IS CHURN ANALYSIS

Customer churn analysis refers to identifying customers who are likely to stop using a company's product or service. It helps in :

- Understanding why customers leave.
- Predicting future churn.
- Creating retention strategies.

TOOLS & TECHNOLOGIES USED

- Jupyter Notebook: To execute code and display results.
- Python: A programming language for data analysis.
- Libraries Used: Numpy, Pandas, Matplotlib, and Seaborn.
- CSV Dataset: "Customer Churn.csv"

BUSINESS REQUIREMENTS

1. What percentage of customers have churned?
2. Is churn related to gender or age group (senior citizen)?
3. How does customer tenure affect churn?
4. Which contract type leads to higher churn?
5. Does the use of specific services (e.g., online security, tech support) impact churn?
6. Do payment methods influence churn rate?
7. Are there data quality issues that need cleaning before analysis?

Importing Required Libraries

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

"We import essential Python libraries for data loading, manipulation, and visualization."

Loading the Dataset and Displaying Initial Rows

```
path="/content/Customer Churn.csv"
df = pd.read_csv(path)
df.head()
```

...

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Internet
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	

“We load the churn dataset into a pandas DataFrame to begin our analysis and preview the first few records to understand the structure and data types.”

Inspecting Data Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
```

“Checking column data types, non-null values, and overall structure of the dataset.”

Handling Null and Inconsistent Data

Replacing blanks with 0 as tenure is 0 and total charges are recorded and converted the datatype from object to float

```
df["TotalCharges"] = df["TotalCharges"].replace(" ","0")
df["TotalCharges"] = df["TotalCharges"].astype("float")
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
```

“Fixing missing or incorrectly formatted values in the ‘TotalCharges’ column.”

Checking for Null Values

```
df.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0

"Fixing missing or incorrectly formatted values in the 'TotalCharges' column."

Checking for Duplicate Records

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df["customerID"].duplicated().sum()
```

```
np.int64(0)
```

"Verifying that customer records are unique and there are no duplicates."

Descriptive Statistics

```
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

"Getting summary statistics (mean, std, min, max) for numeric columns."

Converting Numeric to Categorical

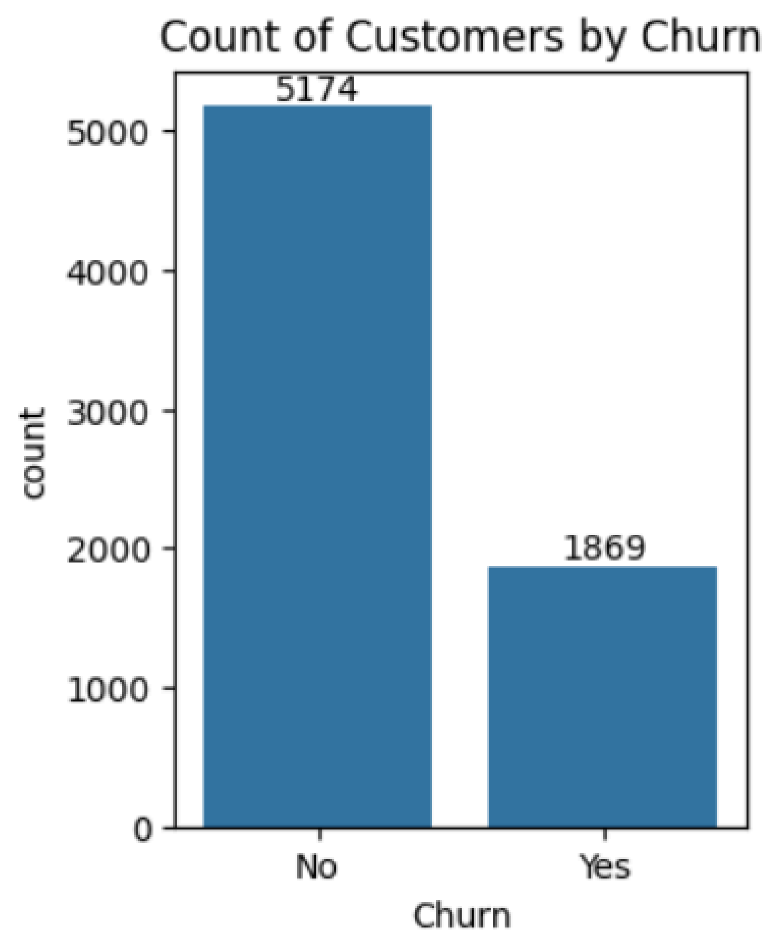
The "SeniorCitizen" column is given as 1 and 0 converted it to yes and no to make it easy to understand

```python
def conv(value):
  if value == 1:
    return "yes"
  else:
    return "no"

df["SeniorCitizen"] = df["SeniorCitizen"].apply(conv)
```

“Making ‘SeniorCitizen’ easier to interpret by converting numeric to 'Yes/No'.”

Count of Churned Customers

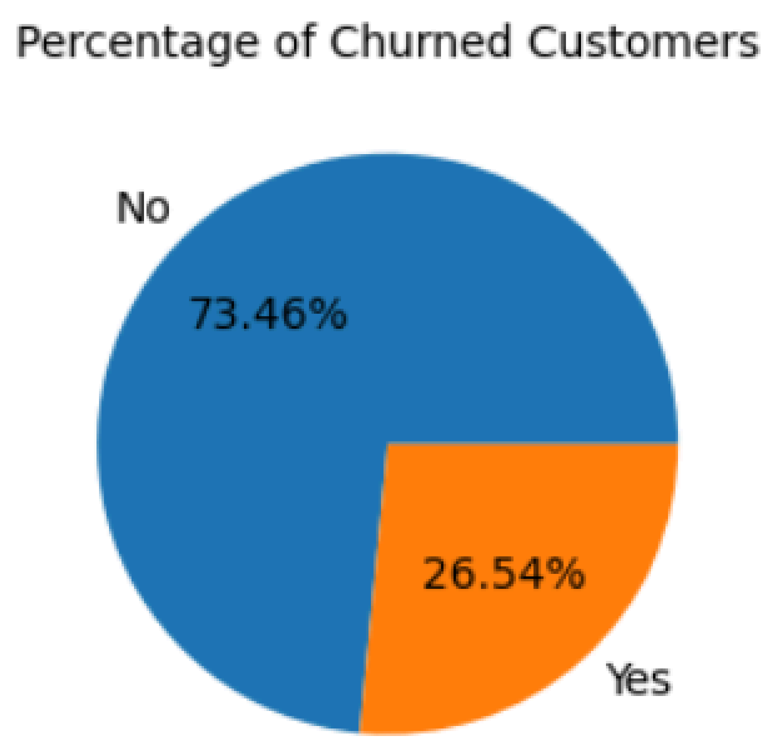
```
plt.figure(figsize=(3,4))
ax = sns.countplot(x = "Churn", data = df)
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Churn")
plt.show()
```



"Visualizing how many customers have churned vs. retained."

CHURN PERCENTAGE

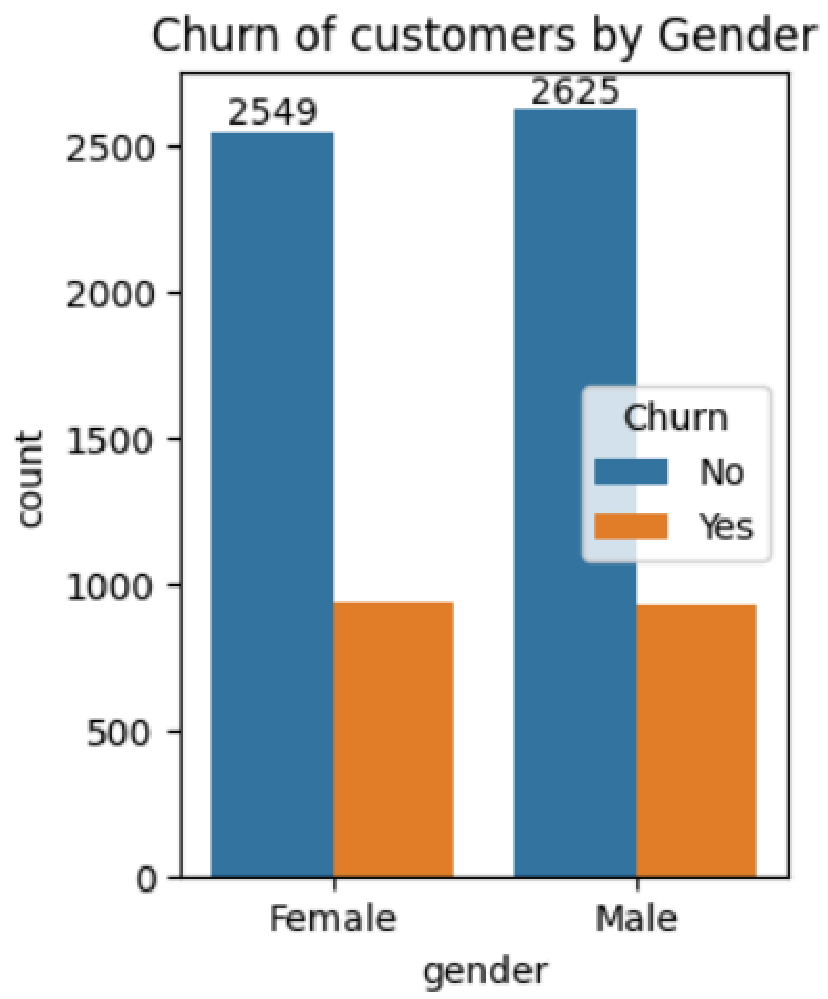
```python
plt.figure(figsize = (3, 4))
gb = df.groupby("Churn").agg({"Churn":"count"})
plt.pie(gb["Churn"], labels = gb.index, autopct = "%1.2f%%")
plt.title("Percentage of Churned Customers", fontsize = 10)
plt.show()
```



"Visualizing the percentage of churned customers using a pie chart."

Churn by Gender

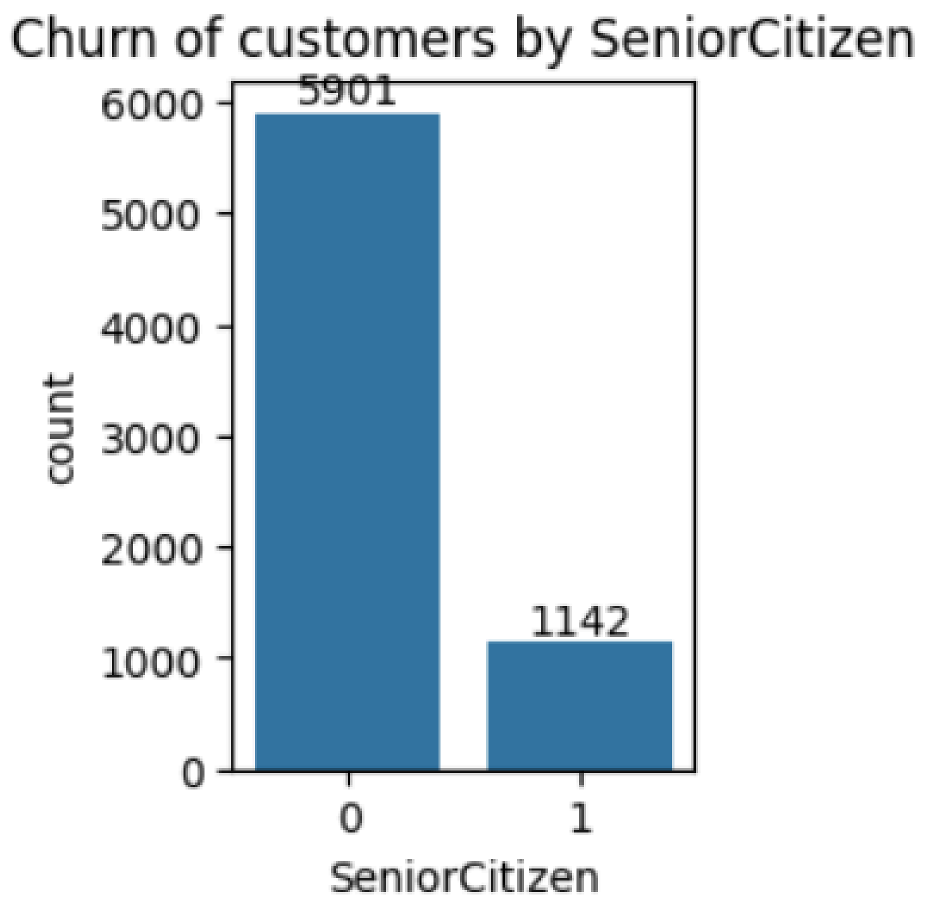
```python
plt.figure(figsize=(3,4))
ax = sns.countplot(x = "gender", data = df, hue = "Churn")
plt.title("Churn of customers by Gender")
ax.bar_label(ax.containers[0])
plt.show()
```



"Analyzing if churn behavior varies by gender."

Count of Senior Citizen

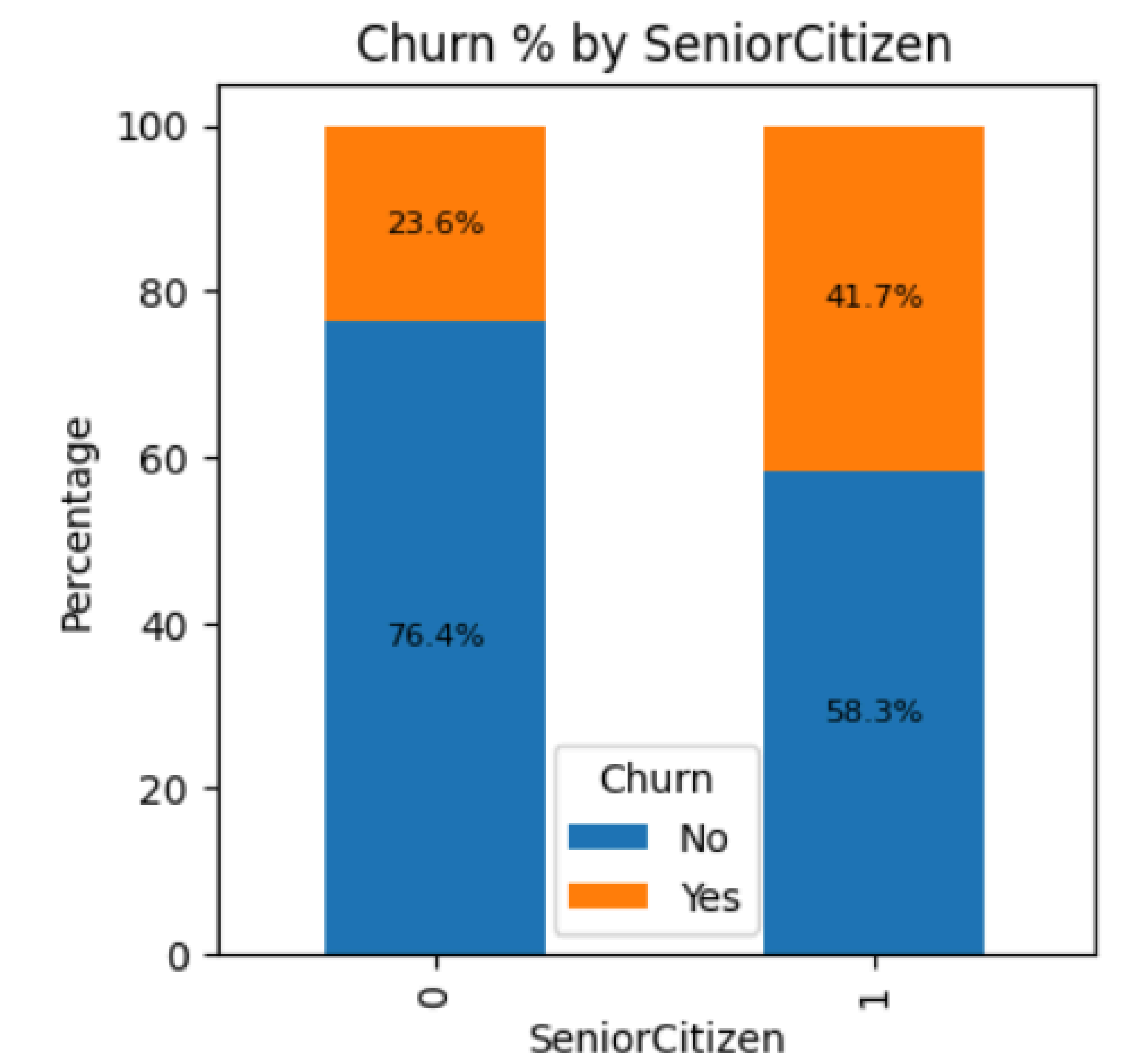
```
plt.figure(figsize=(2,3))
ax = sns.countplot(x = "SeniorCitizen", data = df)
plt.title("Churn of customers by SeniorCitizen")
ax.bar_label(ax.containers[0])
plt.show()
```



"Exploring how many senior citizen customers we have."

Churn Analysis by Senior Citizen

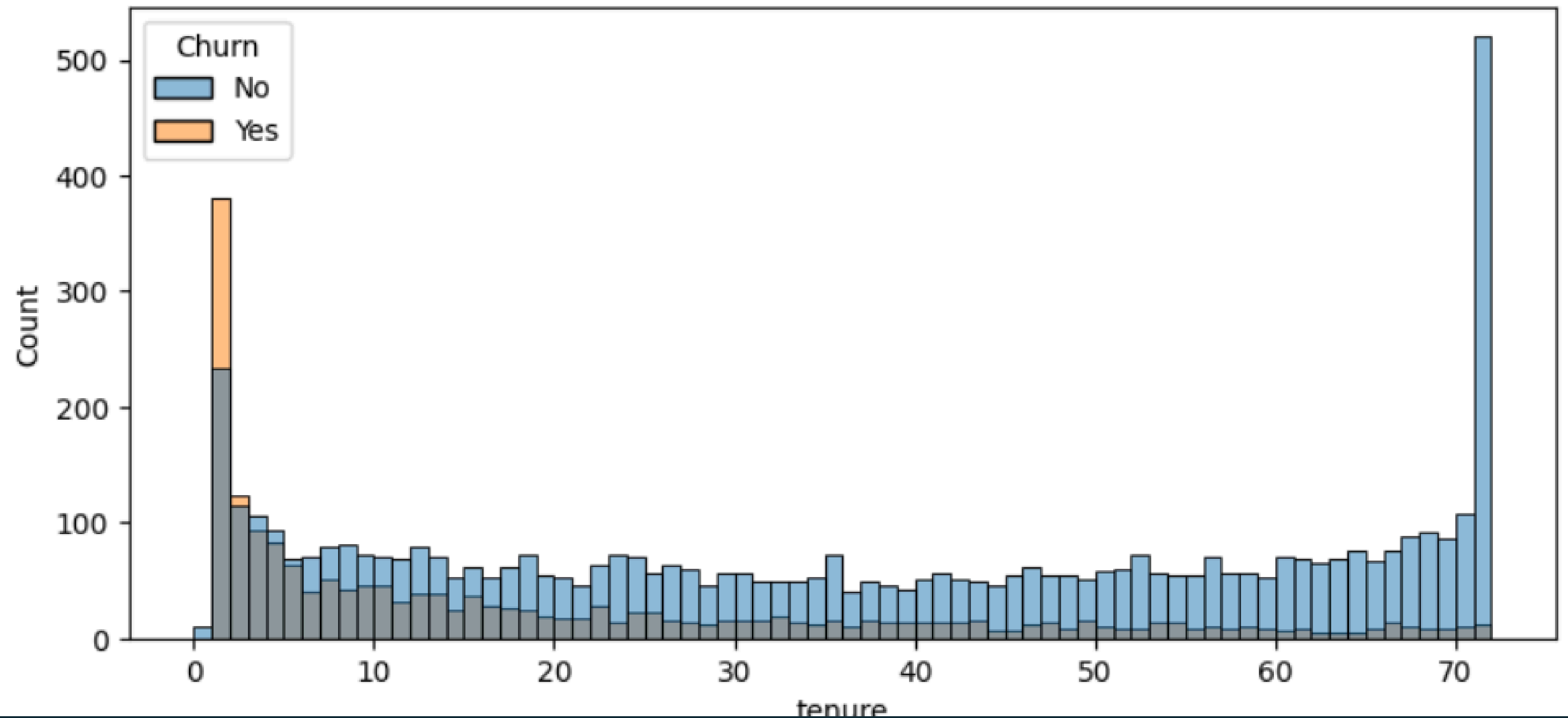
```
plt.figure(figsize=(3,3))

# create crosstab
ct = pd.crosstab(df['SeniorCitizen'], df['Churn'], normalize='index') * 100
ct.plot(kind='bar', stacked=True, figsize=(4,4))

# add % labels
for i, row in ct.iterrows():
    cum = 0
    for churn, val in row.items():
        plt.text(i, cum + val/2, f"{val:.1f}%", ha='center', va='center', fontsize=8)
        cum += val

plt.title("Churn % by SeniorCitizen")
plt.ylabel("Percentage")
plt.legend(title="Churn")
plt.show()
```



"Stacked bar showing churn percentage among senior and non-senior citizens for comparison."

Churn vs. Tenure

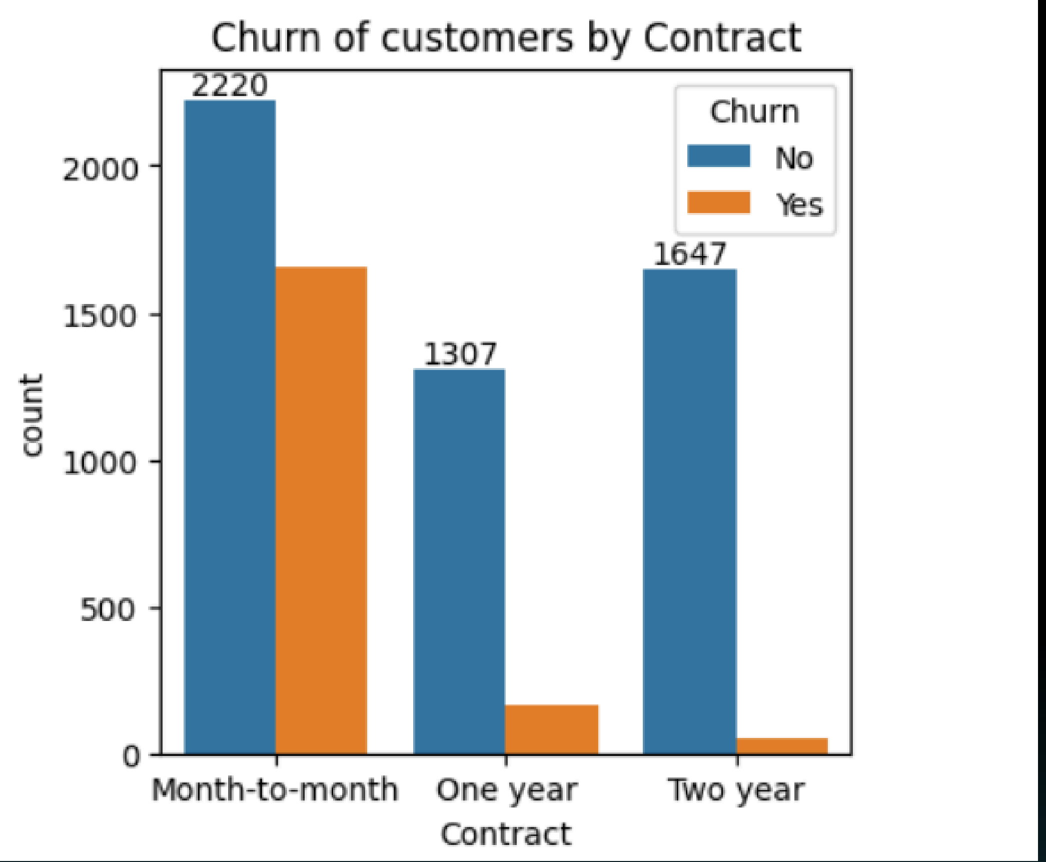
```
plt.figure(figsize = (9,4))
sns.histplot(x = "tenure", data = df, bins = 72, hue = "Churn")
plt.show()
```



"Understanding how contract duration impacts churn."

Churn by Contract Type

```
plt.figure(figsize=(4,4))
ax = sns.countplot(x = "Contract", data = df, hue = "Churn")
plt.title("Churn of customers by Contract")
ax.bar_label(ax.containers[0])
plt.show()
```



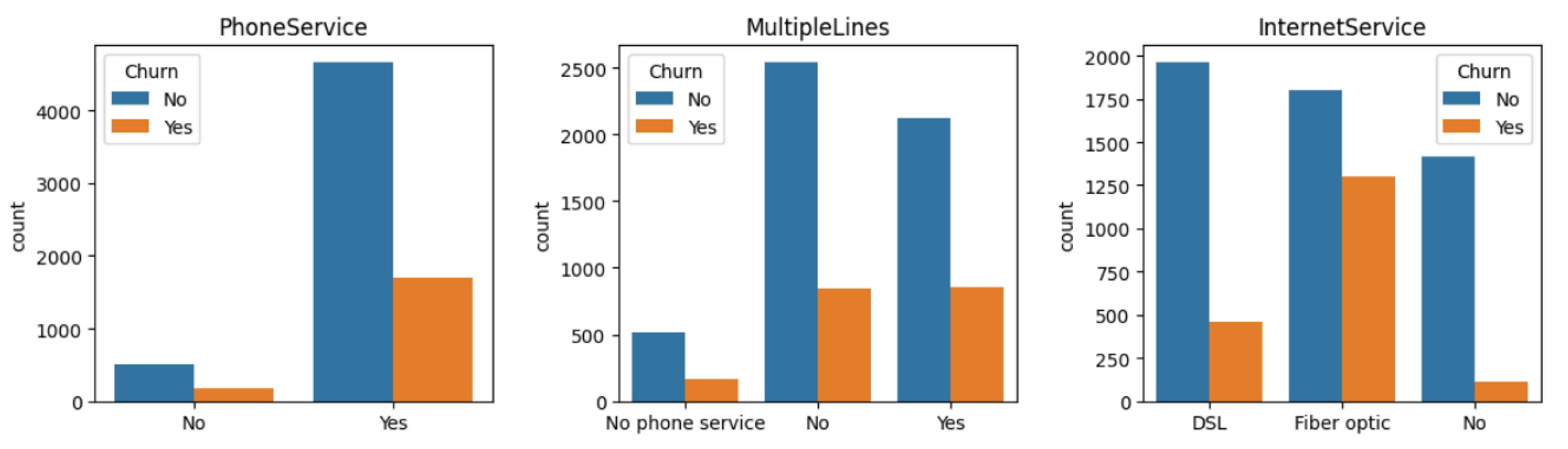
"Churn analysis by contract type to see which contract leads to higher customer loss.

Churn by Available Services

```python
cols = ['PhoneService','MultipleLines','InternetService',
        'OnlineSecurity','OnlineBackup','DeviceProtection',
        'TechSupport','StreamingTV','StreamingMovies']

plt.figure(figsize=(12,10))

for i, col in enumerate(cols, 1):
    plt.subplot(3, 3, i)
    sns.countplot(x=df[col], hue = df["Churn"])
    plt.title(col)
    plt.xlabel("")
    plt.tight_layout()

plt.show()
```

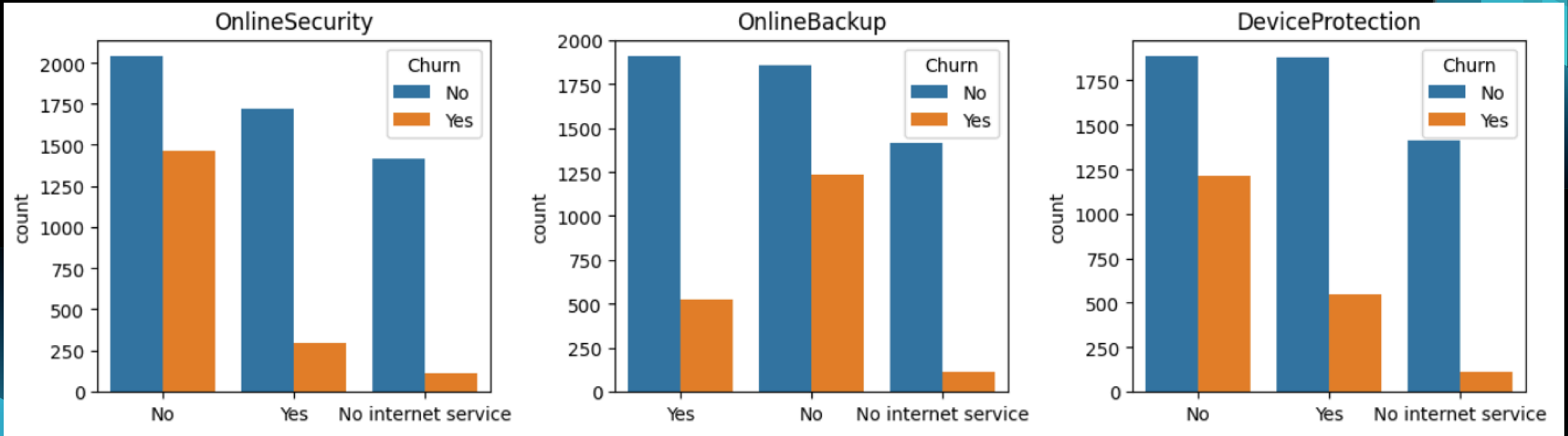
“Churn comparison across various customer services to identify which services are linked to higher churn rates.”

Churn by Available Services



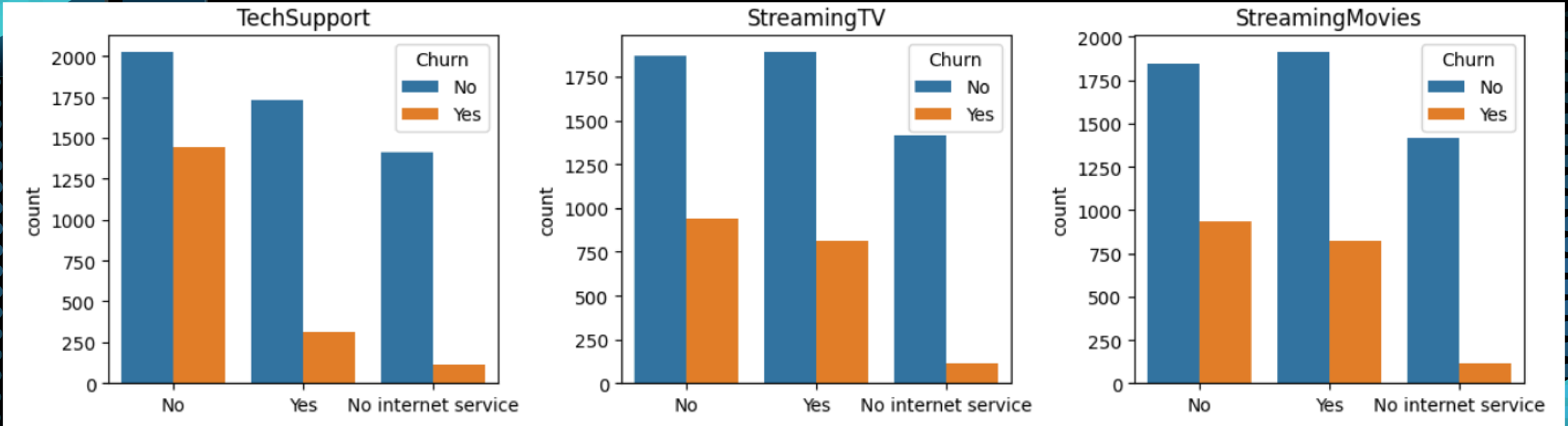
"Churn comparison across various customer services to identify which services are linked to higher churn rates."

Churn by Available Services



“Churn comparison across various customer services to identify which services are linked to higher churn rates.”

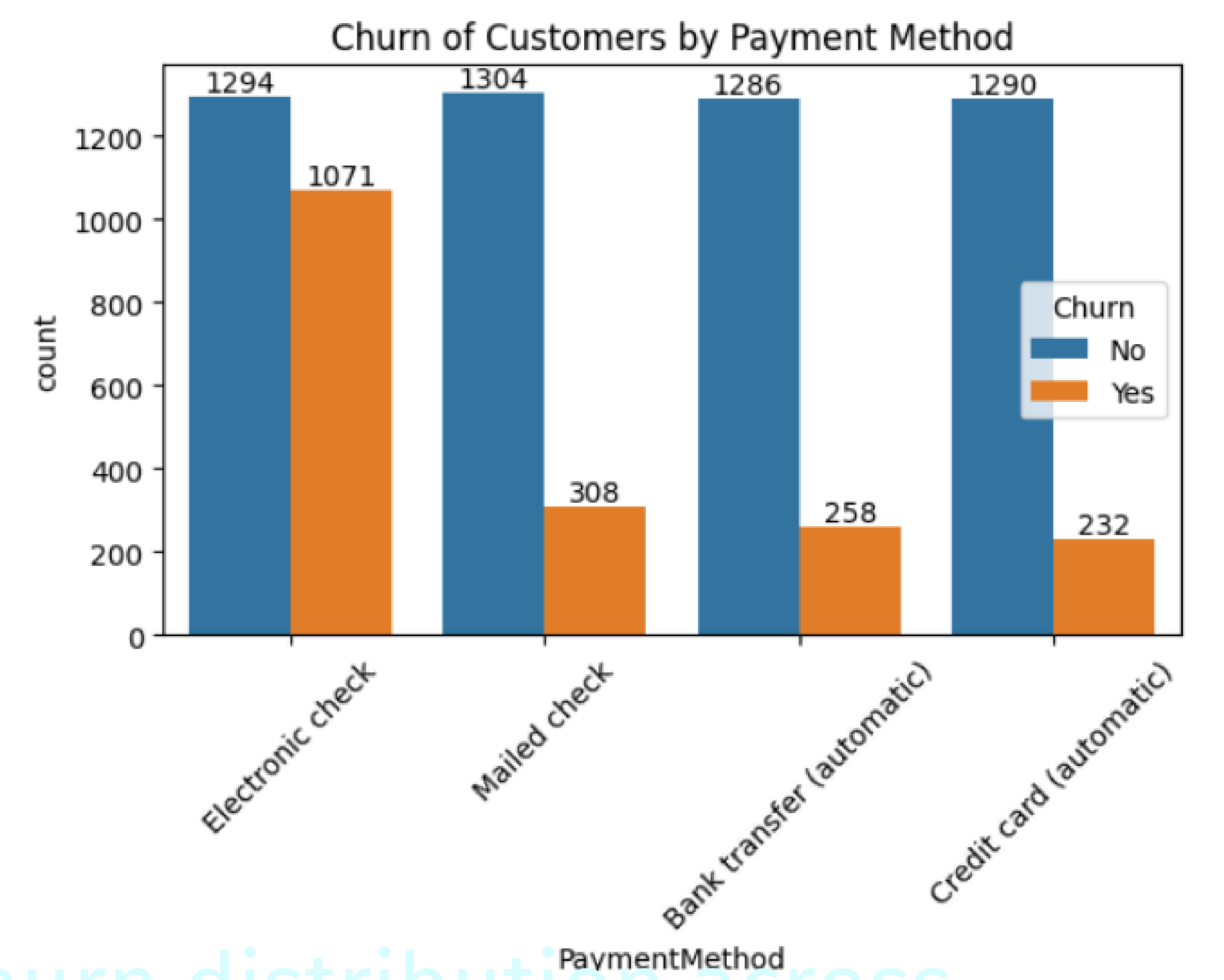
Churn by Available Services



"Churn comparison across various customer services to identify which services are linked to higher churn rates."

Churn by Payment Method

```
plt.figure(figsize=(6,5))
ax = sns.countplot(x="PaymentMethod", data=df, hue="Churn")

ax.bar_label(ax.containers[0])
ax.bar_label(ax.containers[1])

plt.title("Churn of Customers by Payment Method")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



"Grouped bar chart showing churn distribution across different payment methods."

KEY FINDINGS FOR CUSTOMER RETENTION

1. High Churn: A 26.54% churn rate (1,869 customers) demands urgent attention.
2. Gender-Neutral Churn: Churn rates are consistent across genders, suggesting the need for universal retention efforts.
3. Senior Citizen Loyalty: Senior citizens churn less, indicating effective engagement strategies in this demographic.
4. Contract Vulnerability: Month-to-month contracts carry the highest risk of churn; long-term contracts significantly reduce churn.
5. Service Gaps & Churn: Customers are more likely to churn if they lack extra services (like online security or tech help) or if they have fiber-optic internet.
6. Payment Method Risk: Electronic Check users show a higher churn rate, requiring investigation.

RECOMMENDATIONS

1. Prioritize Contract Type: Focus retention efforts on customers on month-tomonth contracts by offering incentives for longer-term commitments
2. Enhance Service Bundling: Promote the adoption of value-added services (OnlineSecurity, TechSupport, etc.) to increase customer stickiness and perceived value.
3. Investigate Fiber Optic Churn: Conduct deeper analysis into Fiber Optic customer feedback and service quality to identify and address pain points.
4. Optimize Electronic Check Experience: Analyze the reasons behind higher churn among Electronic Check users; consider alternative payment method promotions or improving the electronic check process.
5. Leverage Senior Citizen Success: Understand what makes senior citizens less likely to churn and explore if these strategies can be applied to other demographics.

THANK YOU